CPS 350 Assignment: Red Black BST

100 pts

This is **a team project (up to two students per team)**. One submission per team.
Receive 5 bonus points if turn in the complete work at least one day before deadline.
No late submission will be accepted. Receive an F for this assignment if any academic dishonesty occurs

---

1. Purpose

The purpose of this assignment is to implement red-black trees.

2. Description

In this assignment you will do a simple modification based on a given framework of Binary Search Tree (BST) by changing it into a red-black tree in order to maintain the balance. As a binary search tree has a shortage that the tree can grow in an unbalanced manner, which pretty much depends on the order of data (nodes) to be inserted. So your job is to improve the existing software on BST and make it into a balanced version by red-black principle. After you download the provided framework and unzip the file, ***three*** files are extracted: (1) "MainGUI.java" (2) "Node.java" (3) "Tree.java".

2.1. Run the code

(1) Create a "JavaFX project," which is the same as you did on previous JavaFX projects.
(2) Copy the three files above to the source folder of your newly created projects "src/application/" folder. By default, after you create the project, you have "Main.java" and "application.css" files in the folder. You can just delete the "Main.java" (leave the "application.css" as it is) before copying the three files into it.
(3) Then refresh the "application" package in the Eclipse. You should be able to run the code as shown in Figure-1:

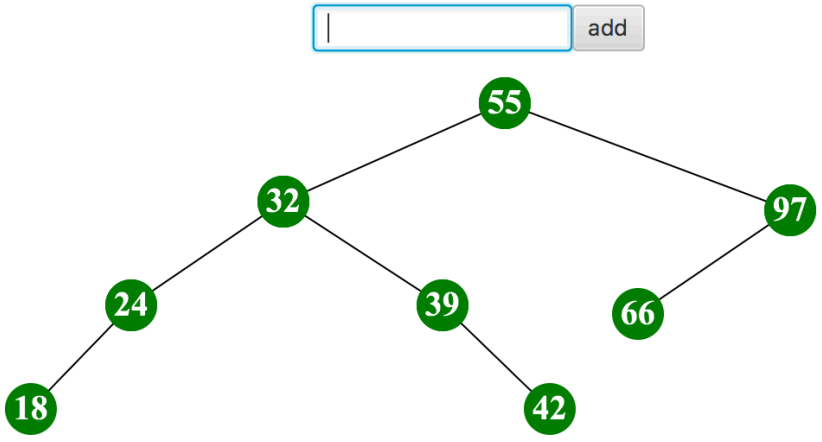


Figure-1: Binary Search Tree User Interface

The GUI allows you to add node into the tree by simply typing a number (with one or two digits only) and hitting “ENTER” key or clicking the “add” button. Then the new node should appear in the tree. Also, you can change the node position freely by simply “dragging” the node on canvas to move it to any preferred position as Figure-2 shows. But it is recommended not to move both the left and right children to one side of the parent or above the parent, which makes the tree structure less meaningful to the concept of BST. Every time when a node is inserted or selected, it is highlighted by a thin “yellow ring” around the node. You can remove such highlight at any time by “Right Clicking” or “Left Clicking” your mouse on any blank area of the canvas. Also, you can use the mouse to choose or highlight any node by left clicking on the node directly.

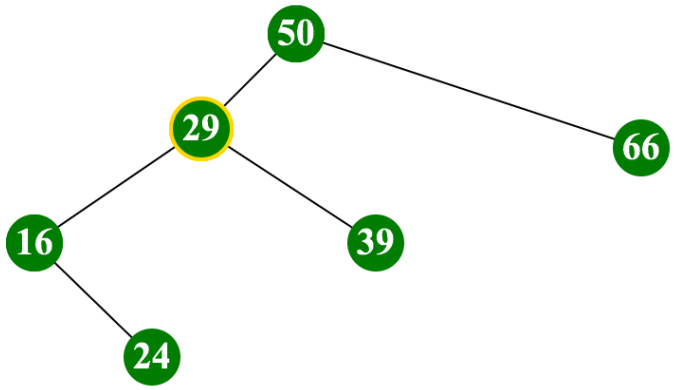


Figure-2: Demo of dragging or moving a node to any position

As discussed earlier, the BST can easily go out of balance. Even with the same set of numbers, the different insertion orders may result in different tree appearances as Figure-3 demonstrates:

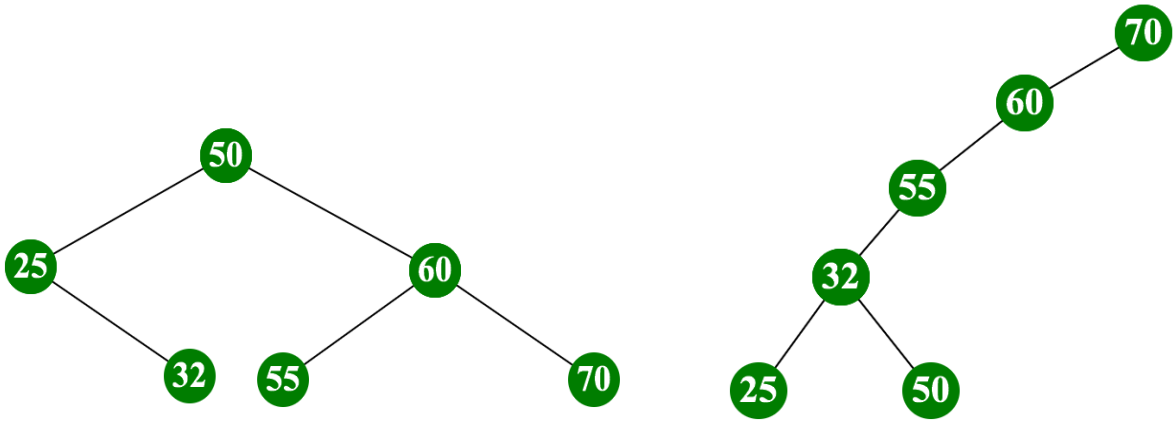


Figure-3: the same set of numbers end up with two tree structures

2.2 Implementation Requirements

So now it is your responsibility to fix this unbalanced issue by placing the red-black tree constraints. In addition, we want that the tree does NOT contain any duplicate keys. So, as the

warm-up tasks, I need you first try two simple things to start coding as describe in (1) and (2) listed below. After finishing these two simple tasks, you can start working on the red-black tree implementation. Below shows the details tasks which all involve modifying the "Tree.java" class.

(1) Modify the "insertNode()" function to make sure no repeating value nodes are inserted, e.g. for the tree in the Figure-3, if you insert 55 again, no any changes should occur on the tree, since 55 already exists.

(2) Try to change node color in the "insertNode()" function by following the example code in the beginning of the function: "node.setColor(Node.*GREEN*);" This code is to set green color. You can also try to change it to "BLACK" or "RED" color as figure 4 shows.

(3) Now, you can start to implement the red-black tree insertion algorithm based on the given "insertNode()" function. Feel free to add more functions if necessary. But the additional function should be called by the "insertNode()" function. In other words, "insertNode()" function is the entry function when an insertion event occurs.

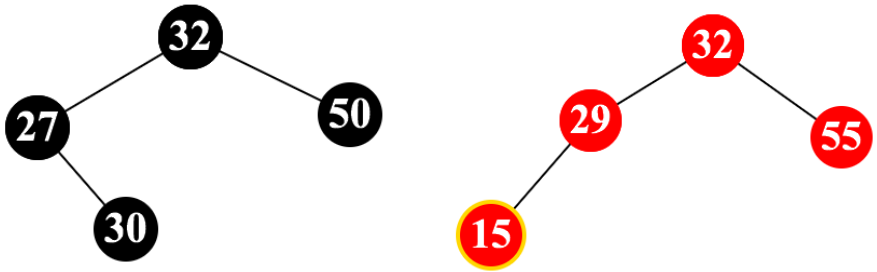


Figure-4: results of changing the tree to other colors. But this is not the final goal for red-black tree, which should have a combination of both red and black colors.

2.3. Add your code

With the existing framework, you need to focus on modifying the code in the "Tree.java" file. You do NOT need to do any change on the "MainGUI.java". However, you may write some code (optional) for the "Node.java" depending on your specific red-black tree implementation. For the "Node.java" and "Tree.java" files, each file has a pair of comment lines:

```
/********************    Implementation Here:  **************************/

/********************    End of Implementation  **************************/
```

You are required to write your code between these two lines. It is NOT encouraged to write or modify any code outside this range.

Hints:

(1) Please follow the comment in the code, which describes the framework and the part for your implementation in details.

(2) For the "Node.java", the data members you may use for the assignment are separated from the ones for the GUI part. So the data members you need to know are at the top of the class, such as "left", "right", "color", etc. In fact, for this class, you just need to use these data members and do not need any additional coding.

(3) For the given code in the "insertNode(Node node)", you may notice other than assigning "left" or "right" child, the existing code also assigns other relationship such as "parent" and "left_child_of_parent" variables. These variables are important as they allow the nodes to be accessed easily across the tree. The GUI also needs this information to draw the tree on the canvas. So when you modify the code, make sure set these variables accordingly. Moreover, the existing code shows the non-recursive way to insert a new node onto a BST. **Your implementation of insertion for a red-black BST should be done recursively, as we show in class**; the recursive method is elegant and precise.

(4) Make sure that **the root is black**. Make sure that you have a method such that it returns true if the node's color is red; **if the node is null, return false**. **This is extremely important; otherwise you may encounter null pointer errors!**

2.4. Rubrics

Though there is only one major function "insertNode()" involved for the red-black tree implementation, the function should conduct a sequence of tasks, e.g. left-rotation or right-rotation, tree balance check. It is encouraged to put these actions into separate sub-functions, which makes your code more organized and easier for grading. Figure-5 shows an example of successful implementation of "insertNode()", which yields a balanced tree with correct color assignment.

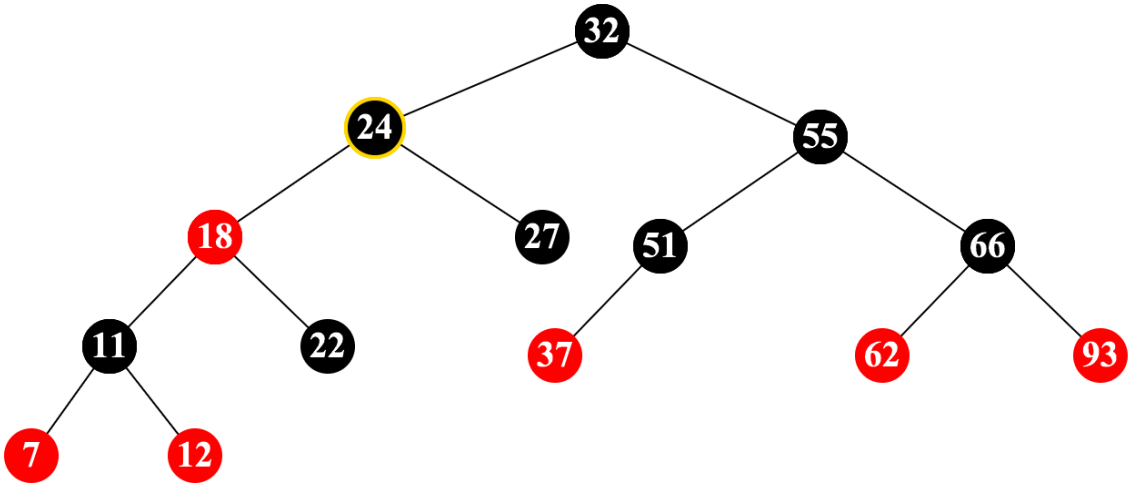


Figure-5: Result of Red-Black tree with balanced node distribution.

3. Grading notes

If your program does not compile, you receive zero points for that program.

(1) (10pts) Report:
- (5pts) Show at least 5 sample runs.
- (5pts) How did you use AI to assist you in this project? List any difficulties you have during implementation. What is the fun part of this assignment?

(2) (10pts) Modify the insertNode() so that the tree does NOT allow a node with repeating value to be inserted.

(3) (80pts) Successfully complete the "insertNode()" function to achieve red-black tree insertion:
   (a) Correct node color assignment (20pts)
   (b) Correct left/right rotation (30pts)
   (c) Correct tree balance (30pts)

4. Submission

Your project should be submitted to canvas. Turn in your updated version of the source codes "**Node.java**" and "**Tree.java**", which should be zipped into a single folder. Submit a **report** for question 1 in 3 Grading notes as a **separate file**.